

Yuno — recorded demo

Auto-driven 4:42 walkthrough of every requirement in the challenge brief. Captions baked into the video pixels, narrator voice mixed in via OpenAI TTS. Recorded with Playwright against the running stack — every API call, run, tool call, approval gate, and cost-cap trip is real.

File

`yuno-demo.mp4` — 8.0 MB · 4:42 · 1440×900 · H.264 / yuv420p / 25 fps · `+faststart` AAC narrator track (alloy voice, 128 kbps mono) mixed with `adelay` at each caption's precise timestamp.

Brief-requirement coverage map

Every functional requirement and "Other Requirements" line from the challenge PDF:

Brief line	Where in the video
Create AI agents (personality, tools, schedules, memory, limits)	§1 cockpit roster · §8 agents list · §8b Ana detail (all fields visible)
Connect them into collaborative workflows	§2 Cited Research · §3 PRD with Approval
Real runtime, execute real tools	§2 (web_search TOOL node) · §6 (http_request via Telegram)
Agents communicate with each other to complete tasks autonomously	§3 (Pip → Mara → Brie pipeline) · §1b (Jarvis delegates)
At least one agent reachable through external messaging channel	§5 (binding) · §6 (real round-trip)
Web UI for managing everything visually	every section
Agents communicate asynchronously	§3 (workflow pause / resume) · §1b (chat is async over WS)
Message history persisted and visible in the UI	§6 (Telegram conversation reload from DB)
At least one agent connected to Telegram	§5 + §6
Chosen runtime actually executes agent logic (not a UI mockup)	§2 real Tavily search + real LLM cost ticks
Agent CRUD: name, role, system prompt, model, tools, channels	§8 list + §8b Ana edit form
Agent configuration: schedules, memory, skills, interaction rules, guardrails	§8b — Soul, persona, model, tools, memory, guardrails all visible
Visual workflow builder with conditions and feedback loops	§3b Draft & Critique opened in React Flow builder — <code>iteration_count < 4</code> loop condition called out
At least 2 pre-built workflow templates	§2 shows the 5 seeded templates
External channel integration: WhatsApp, Telegram, or Slack	§5 + §6 Telegram

Brief line	Where in the video
Live monitoring with real-time logs, inter-agent messages, and token/cost tracking	§2 timeline animates step-by-step with cost
Working end-to-end demo with 2+ agents executing a real task	§2 (3-agent chain) · §3 (4-step with human gate)

Section-by-section timestamps

t≈	§	Beat
0:00	1	Cockpit panorama — gauges, constellation, mission queue
0:10	1b	Talk to Jarvis live — type a prompt, watch the constellation work, reply lands inline
0:36	2	Cited Research — TOOL node (<code>web_search</code>) + 3 agents (Remy → Ana → Brie)
1:42	3	PRD with Approval — Pip drafts → human gate → approve from cockpit → Mara + Brie resume
2:24	3b	Visual workflow builder — Draft & Critique opened in React Flow, palette + canvas + inspector; condition-on-edge feedback loop
2:48	4	Cost guardrail — <code>max_cost_usd=\$0.0001</code> , every call blocked with the exact diagnostic, \$0 spent
3:06	5	Channels — Telegram <code>@jarv_m1_bot</code> ACTIVE
3:14	6	Real Telegram round-trip — historical Jarvis conversation: user DM'd a URL, <code>http_request</code> tool, reply back
3:33	7	Team channels — <code>#growth / #product / #research</code>
3:45	8	Agents — search filter ("research")
4:05	8b	Agent config form — Ana the Analyst's full surface: identity, instructions, soul, persona, tone/traits/values, model, temperature, tools, memory strategy, guardrails
4:27	9	Final cockpit — SPEND, TOKENS have moved

How it was made (it's three pieces)

```
scripts/      ← in /tmp/yuno-screens during the demo session
├─ demo.mjs   Playwright script – drives the UI, injects captions
               as a fixed-position frosted-glass DOM overlay,
               logs (timestamp, caption_text) to captions.json
└─ narrate.mjs Reads captions.json → POSTs each text to OpenAI TTS
               (model=tts-1, voice=alloy, speed=1.05) → builds an
               ffmpeg filter graph with adelay per clip → mixes
               one AAC track + the WebM video into the final MP4
```

Re-record with:

```
make up && make seed          # ensure system is ready
node /tmp/yuno-screens/demo.mjs # records WebM + captions.json
node /tmp/yuno-screens/narrate.mjs # TTS + mix → final MP4
```

Recorded against

- `LLM_MODE=live` — real Anthropic + OpenAI keys, ModelRouter routes by `task_type`
- Real Tavily web search
- 17 agents, 5 workflow templates, 3 team channels (from `scripts.seed`)
- The real `@jarv_m1_bot` Telegram binding with existing conversation history

Note on §4 (cost guardrail design)

`CostCapInterceptor` is a **graceful pre-flight blocker**, not a hard run-killer. Before each LLM call it estimates cost; if `run.total_cost + estimate > cap`, it returns a synthetic `[blocked: cost cap ...]` response. Steps "complete" cleanly, run-level cost stays at \$0, and the operator gets a precise diagnostic on every step ("spent \$0, est +\$0.0046 exceeds \$0.0001 cap"). A harder fail-and-stop mode would be a one-line policy change in the interceptor.